

Graph Algorithms

3. P and NP

(second part)

Martin Milanič
martin.milanic@upr.si
UP FAMNIT

November 6, 2025



In the previous lecture, we started with Chapter 3: P and NP.

Contents:

- Motivation
- Decision problems
- Classes P, NP, and co-NP
- NP-hard and NP-complete problems
- Polynomial reductions, proving NP-completeness, two exercises
- Polynomially equivalent problems, Cook reductions (today)

A quick recap of NP-completeness

P: the set of decision problems that can be solved efficiently
(= by a polynomial-time algorithm).

Examples:

- Given a graph G , two vertices s, t , and a positive integer k , does G contain an s, t -path of length at most k ?
- Given a graph G , is G bipartite?

NP: the set of decision problems for which we can **quickly** verify a positive answer if we are given a “certificate”.

What is a certificate? Whatever helps us verify that the answer is indeed positive, for example a set of vertices or edges that we’re looking for.

Note:

All it takes to prove that a problem is in NP is to *describe* a right certificate.
(We don’t need to show anything about how to *compute* one.)

Examples:

- Given a graph G and an integer k , does G contain an independent set of size at least k ?
- Given a graph G , does G contain a Hamiltonian cycle
(= a cycle through all the vertices)?

Similarly:

co-NP: the set of decision problems for which we can quickly verify a **negative** answer if we are given a “certificate”.

Example: Given a graph G , is it true that all maximal independent sets in G have the same cardinality?

A problem is **NP-hard** if it is at least as difficult as all problems in NP.

- That is, if the existence of a polynomial-time algorithm for the problem would imply the existence of a polynomial-time algorithm for all problems in NP.
- Note: NP-hard problems are not necessarily decision problems. They may also be optimization problems or search problems.

NP-complete problem: a problem that is both in NP and NP-hard.

- Hundreds of NP-complete problems are known.
- To prove that a problem is NP-complete, we use the concept of a **polynomial reduction**: a polynomial-time algorithm that maps instances of one problem to instances of another so that the answer (YES or NO) is preserved.

The general procedure for proving NP-completeness of a given problem is as follows:

- 1) We show that the problem is in NP.
- 2) We make a polynomial reduction from some NP-complete problem to our problem.

We illustrated the procedure with several examples.

- SATISFIABILITY $\propto$ INDEPENDENT SET
- INDEPENDENT SET $\propto$ CLIQUE
- INDEPENDENT SET $\propto$ VERTEX COVER
- VERTEX COVER $\propto$ DOMINATING SET

Cook Reductions

The theory of NP-completeness is based on **decision problems**.

Often, however, we are dealing with **optimization or search problems**.

For example:

- Given a graph G , compute the maximum size of an independent set in G .
- Given a graph G , compute a largest independent set in G .

Such problems cannot be NP-complete, but they can be NP-hard.

To prove NP-hardness, we use a generalization of the concept of polynomial reduction, the so-called **Cook reduction**.

Recall:

Definition

A problem Π_1 is **polynomially reducible** to a problem Π_2 if for every instance I for Π_1 we can construct in time polynomial in the size of I an instance $J = J(I)$ for problem Π_2 such that the answer to Π_1 given I is the same as the answer to Π_2 given J .

Reductions as above are also referred to as **Karp reductions**.

A less restrictive type of reduction is often used, called **Cook reduction**.

Definition

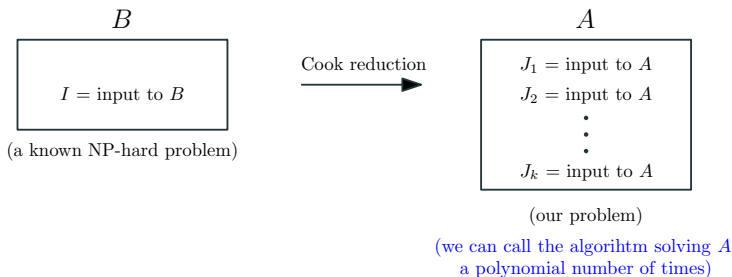
A problem Π_1 is **Cook reducible** to a problem Π_2 if the existence of a polynomial-time algorithm for problem Π_2 implies the existence of a polynomial-time algorithm for problem Π_1 .

Informally: if we have a “black box” at hand that efficiently solves Π_2 , then we can efficiently solve Π_1 using the box.

- Karp reductions are useful only for decision problems.
- Cook reductions are useful also for optimization and search problems.

If we want to prove that a problem A is **NP-hard**, we find a problem B that is known to be NP-hard and

show that if we are given a polynomial-time algorithm for A , we can use it to solve B in polynomial time.



We make a Cook reduction from problem B to problem A .

Example:

Prove that INDEPENDENT SET AUGMENTATION is NP-hard.

$A =$ INDEPENDENT SET AUGMENTATION: Given a graph G and an independent set I , find an independent set I' such that $|I'| > |I|$

(or determine that no such set exists).

For B we take:

$B =$ INDEPENDENT SET:

Given a graph G and an integer k , does G contain an independent set of size at least k ?

The reduction from B to A is given by the following algorithm for solving B using an algorithm for A :

$I = \emptyset$;

while (G has an independent set I' such that $|I'| > |I|$)

$I \leftarrow I'$

end while

if $|I| \geq k$ **return** YES;

else return NO.

B : INDEPENDENT SET

(G, k) = input to B

(a known NP-hard problem)

Cook reduction

inputs to A

A : INDEPENDENT SET AUGMENTATION

(G, I_0) , with $I_0 = \emptyset$

(G, I_1) , with I_1 = the set computed
after the first iteration of the **while** loop

$\vdots$

(G, I_j) , with I_j = the set used as input
for the last iteration of the **while** loop

(our problem)

Polynomially equivalent problems

If two problems are Cook reducible to each other, we say that they are **polynomially equivalent**.

Example:

For the INDEPENDENT SET problem the following three variants of the problem are pairwise polynomially equivalent:

- **Decision** variant:
Given a graph G and a positive integer k , does G have an independent set of size at least k ?
- **Optimization** variant:
Given a graph G , find $\alpha(G)$ = the **independence number** of G = the maximum size of an independent set in G .
- **Search** variant:
Given a graph G , find an independent set of maximum size in G .

Clearly, if there is a polynomial-time algorithm for the search variant, then there is a polynomial-time algorithm for the optimization variant.

- Given an independent set S of maximum size, just return the size of S .

Similarly, if there is a polynomial-time algorithm for the optimization variant, then there is a polynomial-time algorithm for the decision variant.

- G has an independent set of size at least k if and only if $\alpha(G) \geq k$.

Suppose that there is a polynomial-time algorithm for the optimization variant.

Then, the following recursive algorithm ALPHA-SEARCH solves the search variant in polynomial time:

```
ALPHA-SEARCH
if  $V(G) = \emptyset$  then
    return  $\emptyset$ ;
else let  $v \in V(G)$ ;
    if  $\alpha(G) = \alpha(G - v)$  then
        return ALPHA-SEARCH( $G - v$ );    //  $v$  can be safely deleted
    else //  $v$  has to be selected
         $S \leftarrow$  ALPHA-SEARCH( $G - N[v]$ );
        // delete  $v$  and all its neighbors
        return  $S \cup \{v\}$ ;
    end if
end if
```

Exercise: prove the correctness of the above algorithm and analyze its running time, assuming that the computation of the independence number of an n -vertex graph takes time $\mathcal{O}(n^c)$ for some fixed positive integer c .

Finally, assume that there is a polynomial-time algorithm for the decision variant.

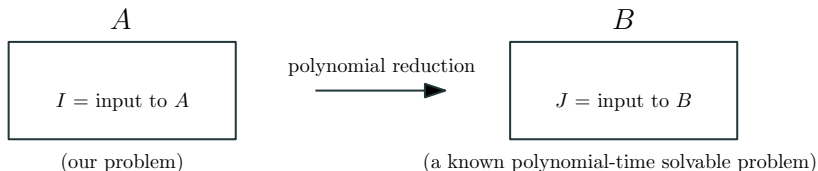
Then, the following algorithm solves the optimization variant in polynomial time:

```
for  $k = |V(G)|, |V(G)| - 1, \dots$  downto 1  
    if  $G$  has an independent set of size at least  $k$  then  
        return  $k$ ;  
    end if  
end for
```

Similar equivalencies hold for several other graph problems.

Summary of using Karp and Cook reductions

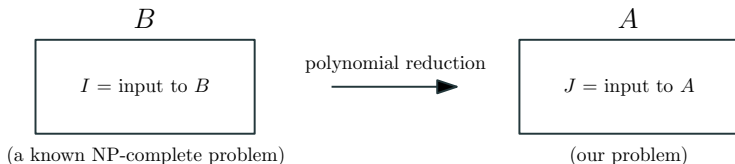
If we want to prove that a decision problem A is **polynomial-time solvable**, we find a problem B that is known to be polynomial-time solvable and **reduce (= find a polynomial reduction from) A to B** .



B could be, for example, any of the classical polynomial-time solvable problems (2-SATISFIABILITY, or the decision version of: shortest paths, spanning trees, maximum flow, vertex cover in bipartite graphs, maximum matching, etc.).

If we want to prove that a problem A is **NP-complete**, we do two things:

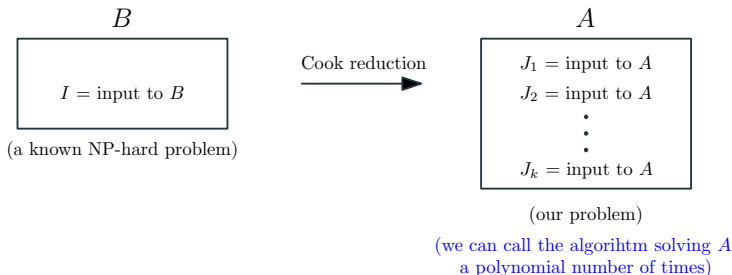
1. verify that A is in NP (this is usually the easy part – we just need to *describe* a suitable certificate and argue that it can be used to verify in polynomial time that we have a yes instance) and
2. find a problem B that is known to be NP-complete and **reduce (= find a polynomial reduction from) B to A .**



B could be any of the known NP-complete problems, for example SATISFIABILITY, INDEPENDENT SET, VERTEX COVER, CLIQUE, DOMINATING SET.

If we want to prove that a problem A is **NP-hard**, we find a problem B that is known to be NP-hard (for example, one of the known NP-complete problems) and

show that if we are given a polynomial-time algorithm for A , we can use it to solve B in polynomial time.



Similar reductions (in the opposite direction) are useful if we want to prove that a problem A is **polynomial-time solvable**.

Let us revise a known example.

Example:

Let A be the following problem:

Given a graph G , compute the vertex connectivity $\kappa(G)$.

For B we take the maximum flow problem.

We showed in class how to reduce A to B :

COMPUTING THE VERTEX CONNECTIVITY OF A GRAPH

Input: A graph $G = (V, E)$.

Output: $\kappa(G)$.

if G is a complete graph **then**

return $\kappa(G) = |V| - 1$;

$D :=$ digraph obtained from G by replacing each edge with a pair of oppositely oriented directed edges.

for all $\{s, t\} \in \binom{V}{2} \setminus E$

$D_{st} :=$ digraph obtained from D as in the proof of implication

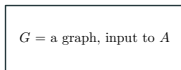
$(c) \Rightarrow (d)$ of Menger's Theorem;

$\kappa_G(s, t) :=$ the maximum value of a flow in the network $(D_{st}, \mathbf{1}, s'', t')$;

end for

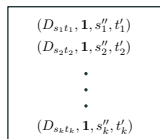
return $\kappa(G) = \min \left\{ \kappa_G(s, t) : \{s, t\} \in \binom{V}{2} \setminus E \right\}$;

A : Given G , compute $\kappa(G)$



Cook reduction $\rightarrow$

$B =$ the maximum flow problem



$(k = \mathcal{O}(|V(G)|^2))$



How to Deal With NP-complete Problems?

There are several general approaches on how to deal with the intractability of NP-complete problems:

- polynomial algorithms for particular input instances
- approximation algorithms
- heuristics, local optimization
- “efficient” exponential algorithms (e.g., 1.5^n instead of 2^n)
- randomized algorithms
- parameterized complexity
(fixed-parameter tractable (FPT) algorithms)

We will discuss some of them in the upcoming lectures.

**Solutions to two exercises
posed at the end of previous lecture**

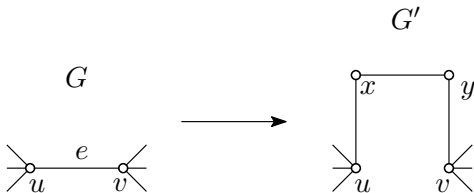
1. For a graph G let $\tau(G)$ denote the minimum size of a vertex cover in G .

(a) Show that replacing a single edge in G with a three-edge path increases the value of τ by precisely 1.

Formally: if $G = (V, E)$ is a graph and $e = uv \in E$, then

$$\tau(G') = \tau(G) + 1,$$

where $V(G') = V(G) \cup \{x, y\}$ (vertices $x, y \notin V(G)$ are two new vertices) and $E(G') = (E(G) \setminus \{e\}) \cup \{ux, xy, yv\}$.



Solution:

We need to show that $\tau(G') \leq \tau(G) + 1$ and $\tau(G') \geq \tau(G) + 1$.

- $\tau(G') \leq \tau(G) + 1$.

Let C be a smallest vertex cover in the graph G .

Since C is a vertex cover, $u \in C$ or $v \in C$ holds.

Without loss of generality, we may assume that $u \in C$.

Then $C \cup \{y\}$ is a vertex cover in the graph G' .

It follows that $\tau(G') \leq |C \cup \{y\}| = |C| + 1 = \tau(G) + 1$.

- $\tau(G') \geq \tau(G) + 1$.

We will show the equivalent inequality $\tau(G) \leq \tau(G') - 1$.

Let C' be a smallest vertex cover in the graph G' .

Since C' is a vertex cover, $x \in C'$ or $y \in C'$ holds.

Without loss of generality, we can assume that $x \in C'$.

We can also assume that $v \in C'$. Namely, if $v \notin C'$, then $y \in C'$ (since otherwise the edge yv would not be covered) and C' could be replaced by the vertex cover $(C' \setminus \{y\}) \cup \{v\}$ which has the same cardinality as C' and contains v .

Let $C = C' \cap V(G)$.

From the assumption $v \in C'$ it follows that $v \in C$. Then C is a vertex cover in the graph G and therefore

$$\tau(G) \leq |C| \leq |C' \setminus \{x\}| = |C'| - 1 = \tau(G') - 1.$$

(b) A **triangle** in a graph G is a clique of size 3.

Using the result in part (a) show that the VERTEX COVER problem is NP-hard already for graphs without triangles.

Hint: Make a reduction from VERTEX COVER.

Solution:

VERTEX COVER for graphs without triangles is in NP, since it is in NP even for general graphs.

To prove NP-completeness, we follow the hint and make a reduction from VERTEX COVER.

$I = (G = (V, E), k)$ instance of VERTEX COVER $\mapsto$

$J(I) = (G', k')$ instance of VERTEX COVER for graphs without triangles.

The graph G' is the graph obtained from G by replacing each of the edges with a path with 4 vertices (as in part (a) of the problem).

For k' we take $k + |E|$.

We note that G' is a triangle-free graph.

Because of part (a), $\tau(G') = \tau(G) + |E|$ holds.

Consequently, there exists a vertex cover with cardinality at most k in the graph G if and only if there exists a vertex cover with cardinality at most k' in the graph G' .

(c) Using the result in part (b) show that the following problem is NP-complete:

TRIANGLE TRANSVERSAL

Input: A graph $G = (V, E)$, integer k .

Question: Is there a set $C \subseteq V$ that contains
at least one vertex from every triangle in G
and such that $|C| \leq k$?

Solution:

Let us introduce the following terminology. A **triangle transversal** in a graph G is an arbitrary set $C \subseteq V(G)$ containing at least one vertex of each triangle in G .

TRIANGLE TRANSVERSAL is in NP: if the answer is YES, we can take for a certificate any triangle transversal $C \subseteq V$ such that $|C| \leq k$.

To prove NP-completeness, we make a reduction from the VERTEX COVER problem for triangle-free graphs.

$I = (G = (V, E), k)$ instance of VERTEX COVER, where G is a triangle-free graph $\mapsto J(I) = (G', k')$ instance of TRIANGLE TRANSVERSAL.

The graph G' is the graph obtained from the graph G by adding a new vertex x^{uv} for each of the edges $uv \in E$ and making it adjacent to the vertices u and v .

$$k' = k.$$

Let us show that in the graph G there is a vertex cover with cardinality at most k if and only if in the graph G' there is a triangle transversal with cardinality at most k' .

Let C be a vertex cover in the graph G with cardinality at most k .

Then C is a triangle transversal in the graph G' .

Since the graph G is triangle-free, every triangle in the graph G' is of the form $\{u, v, x^{uv}\}$ for some edge $uv \in E$.

Therefore, if the set C were not a triangle transversal in the graph G' , then there would be some edge $uv \in E$ such that $C \cap \{u, v, x^{uv}\} = \emptyset$ and consequently $C \cap \{u, v\} = \emptyset$, contradicting the assumption that C is a vertex cover in the graph G .

Consequently: in the graph G' there exists a triangle transversal with cardinality at most $k = k'$.

Now let C' be a triangle transversal with cardinality at most k' in the graph G' .

If C' does not contain any of the vertices $\{x^{uv} : uv \in E\}$, then for each edge $uv \in E$ the set C' can intersect the triangle $\{u, v, x^{uv}\}$ only in vertices u and v , which means that C' is a vertex cover in the graph G with cardinality at most $k' = k$.

But if C' contains a vertex x^{uv} for some edge $uv \in E$, then C' could be replaced by a triangle transversal $(C' \setminus \{x^{uv}\}) \cup \{u\}$ that does not contain the vertex x^{uv} and whose cardinality is at most that of C' .

By successively repeating this process, we arrive at a triangle transversal that does not contain any of the vertices $\{x^{uv} : uv \in E\}$.

Such a set is a vertex cover in the graph G .

Consequently: in the graph G there is a vertex cover with cardinality at most $k' = k$.

2. Show that the following problem is NP-complete:

CLIQUE OR INDEPENDENT SET

Input: A graph $G = (V, E)$, integer k .

Question: Does G contain a clique of size at least k
or an independent set of size at least k ?

Hint: make a reduction from INDEPENDENT SET.

You can assume the fact that replacing a single edge in G with a three-edge path increases the independence number α by precisely 1 (that is, for the resulting graph G' we have $\alpha(G') = \alpha(G) + 1$).

Solution:

The problem is in NP. If the answer is YES, then there is a clique of cardinality at least k or an independent set of cardinality at least k .

In the first case, we take a clique of cardinality at least k as a certificate, and in the second case we take an independent set of cardinality at least k .

To prove NP-completeness, we follow the hint and make a reduction from INDEPENDENT SET.

$I = (G = (V, E), k)$ instance of INDEPENDENT SET.

$J(I) = (G', k')$ instance of CLIQUE OR INDEPENDENT SET.

The reduction is the same as in the solution to part (b) of the previous exercise. We replace each edge with a path of length 3. The resulting graph is triangle-free.

We also have: $\alpha(G') = \alpha(G) + |E|$.

For k' we take $k + |E|$.

The following statements are equivalent:

- The graph G contains an independent set with cardinality at least k .
- The graph G' contains an independent set with cardinality at least $k' = k + |E|$.
- A graph G' contains a clique with cardinality at least k' or an independent set with cardinality at least k' .

The equivalence between the second and third statements holds because the graph G' is triangle-free. So for every $k' \geq 3$, the graph G' does not contain a clique with cardinality at least k' .

The assumption $k' \geq 3$ is without loss of generality, since for the instance (G, k) for the INDEPENDENT SET problem, we can assume $k \geq 3$.

(For constant values of k , the problem is in P and therefore not NP-hard, unless $P = NP$.)